Resume Screening and Job Description Matching System

AI/ML Specialisation Capstone Project Problem Statement

Field	Details
Project Number	5
Domain	HR Tech / NLP
Recommended Group Size	5 students
Primary Dataset	Resume-JD Match Dataset
Dataset Source URL	https://huggingface.co/datasets/facehuggerapoorv/resume-jd-match
Core Curriculum Alignment	Deep Learning, TensorFlow/Keras, CNN/RNN/Transformers/Generative Models as applicable, Model Optimisation, Deployment, and MLOps.
Expected Prototype	Streamlit dashboard or FastAPI API for JD-resume matching.

1. Background and Context

This capstone project is designed for a group of five learners to demonstrate end-to-end AI/ML capability. It connects model development with data preparation, performance evaluation, deployment readiness, and business interpretation. The project belongs to the HR Tech / NLP domain and uses the Resume-JD Match Dataset dataset.

2. Problem Statement

Recruiters and placement teams need to screen resumes against job descriptions efficiently. Manual screening is time-consuming and may be inconsistent. The project requires learners to build a system that estimates resume-job fit and explains skill gaps.

3. Project Objective

Develop a resume-JD matching system that ranks candidates, predicts fit categories, and highlights relevant skills and missing skills in an interpretable manner.

4. Target Users and Use Case

Recruiters, career services teams, placement teams, HR analysts, and learners preparing for jobs.

5. Dataset Details

Field	Details
Dataset Name	Resume-JD Match Dataset
Dataset Summary	Public resume-job description datasets include pairs of resumes and job descriptions with fit labels or match indicators.
Dataset Access Type	Hugging Face Datasets
Source URL	https://huggingface.co/datasets/facehuggerapoorv/resume-jd-match

Recommended First Step	Download or load a small subset first, validate labels and file structure, then scale the model training pipeline.

6. Steps to Access the Dataset

- Install packages: pip install datasets sentence-transformers transformers scikit-learn.
- Load the dataset using: from datasets import load_dataset; dataset = load_dataset('facehuggerapoorv/resume-jd-match').
- Inspect available fields such as resume text, job description text, and match/fit label.
- Clean text by removing excessive whitespace, boilerplate sections, and non-informative symbols.
- Create train-validation-test splits if the dataset provides only one split.
- Generate sentence embeddings for resumes and job descriptions using a pretrained embedding model.
- Document any synthetic or generated data limitations if present in the dataset card.

7. Scope of Work

In Scope

- Parse and clean resume and JD text.
- Build a semantic similarity baseline using sentence embeddings.
- Train or fine-tune a classifier for fit prediction.
- Generate skill-gap insights using keyword and embedding-based extraction.
- Build a dashboard that accepts a JD and multiple resumes and ranks candidates.

Out of Scope

- Automated hiring decisions.
- Use of protected attributes such as gender, caste, religion, age, or marital status.
- Production integration with ATS systems.

8. Recommended Technical Approach

- Define the problem clearly and convert it into a measurable machine learning task.
- Set up a reproducible project repository with folders for data access scripts, notebooks, trained models, reports, and deployment files.
- Perform dataset validation, exploratory analysis, preprocessing, and train-validation-test split creation.
- Build the recommended modelling pipeline: TF-IDF/cosine baseline, Sentence-BERT embeddings, transformer classifier, optional reranking..
- Track experiments with model configuration, validation metrics, training curves, and error observations.
- Tune key hyperparameters such as learning rate, batch size, number of epochs, optimizer, regularization, and model architecture choices.
- Evaluate on a holdout test set and prepare visual evidence such as confusion matrix, sample predictions, error cases, or mask/box overlays.
- Package the final model into a lightweight demo interface or API and document how to run it.

9. Model Evaluation Plan

Primary evaluation metrics: Accuracy/F1 for fit labels, ranking quality using Precision@K or NDCG where applicable, explanation quality review, fairness checklist.

The group should not report only one headline metric. The evaluation must include overall performance, class-wise or segment-wise performance where relevant, failure cases, latency, and a brief explanation of whether the model is usable for the stated user context.

10. Deployment Expectation

Streamlit dashboard or FastAPI API for JD-resume matching.

- The final prototype should load the trained model artifact without requiring retraining.
- The app/API should accept a user input in the expected format and return model output in a clear format.
- The demo should include at least three sample predictions and one known failure case.
- The README should include setup steps, dependencies, and run instructions.

11. Suggested Team Structure for 5 Students

Role	Responsibility
Student 1 - Project Lead & Business Analyst	Owns problem framing, stakeholder context, scope control, final narrative, and presentation coordination.
Student 2 - Data Engineer	Owns dataset access, data validation, preprocessing pipeline, train-validation-test splits, and reproducibility.
Student 3 - Modeling Lead	Owns baseline model, advanced model implementation, training experiments, and tuning strategy.
Student 4 - Evaluation & Research Lead	Owns metrics, error analysis, fairness/ethics checks, benchmarking, and limitation documentation.
Student 5 - Deployment & Documentation Lead	Owns Streamlit/FastAPI demo, Docker or deployment packaging, user guide, and final report formatting.

12. Expected Deliverables

- Problem statement and scope document.
- Data access and preprocessing notebook/script.
- EDA notebook with key observations and data-quality checks.
- Baseline model notebook and advanced model notebook.
- Model evaluation report with metrics, plots, and sample errors.
- Deployment prototype using Streamlit, FastAPI, or equivalent.
- Final presentation deck with problem, method, results, limitations, and future scope.
- GitHub/project folder with README and reproducible instructions.

13. Success Criteria

- Dataset is accessed from a documented public source and loaded reproducibly.
- The group demonstrates a meaningful baseline and an improved advanced model.
- Evaluation metrics are appropriate for the task and interpreted correctly.
- The deployed prototype works on new sample inputs.
- The final explanation clearly connects technical results to user/business value.
- Limitations, risks, and responsible use considerations are stated honestly.

14. Key Risks and Mitigation

Risk	Suggested Mitigation
Resume datasets may include synthetic or biased	Validate assumptions early, document limitations, and

examples.	use error analysis to guide improvements.
Models may learn proxy bias from education, geography, or experience patterns.	Validate assumptions early, document limitations, and use error analysis to guide improvements.
PDF parsing can introduce noisy text extraction.	Validate assumptions early, document limitations, and use error analysis to guide improvements.
Responsible use concern	This project must be positioned as decision support. It should include bias checks and should never be used as the sole basis for rejecting candidates.

15. Final Presentation Guidance

- Slide 1: Project title, team members, and problem context.
- Slide 2: Dataset source, access method, data size, and sample records/images.
- Slide 3: EDA and preprocessing decisions.
- Slide 4: Baseline model and advanced model architecture.
- Slide 5: Experiment tracking and hyperparameter choices.
- Slide 6: Evaluation results and error analysis.
- Slide 7: Deployment demo and inference workflow.
- Slide 8: Risks, limitations, ethical considerations, and future enhancements.